Let us revisit the database we constructed when we were taking a look at the relational database. Let us take a look, not only at the kind of data it is, and but also the kind of ways such data might be accessed or queried. The assumption that we make now might not be representative of the actual usage of such a system, but are mostly made to illustrate the benefits of non-relational databases.

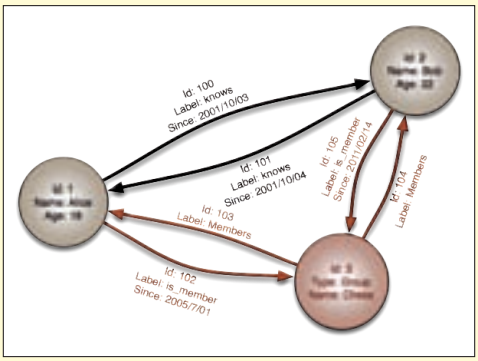
Since we are talking about student scores, we can imagine that the most common use would be to add look up the scores of a particular student, or to add exam scores. While it is important to find out how many students pass or fail, who all take a particular subject, or are eligible for a scholarship; such operations don't need to be performed often.

So when the scores of a particular student are being looked up, it needs to refer to three different tables, combine the details and then only do you get a response. Or four different tables if you have multiple scores per exam.

Also, what if the educational institution maintaining such a database decides at some point, that they need to move from one score per exam to multiple scores per exam? You could add a new table and everything, but handling old scores and new scores at the same time would be difficult.

What if we look at a simple key-value system. The key would be the roll number of the student, and the value would contain all of the student's details and scores. Looking up the scores in this case would be much faster, so would modifying them. Looking for students with scores below a threshold would not be as optimal, however its a relatively infrequent operation and the overall benefits might outweigh the costs. Since there are no tables here, future modifications that involve multiple scores are simple, since both old and new style documents can co-exist.

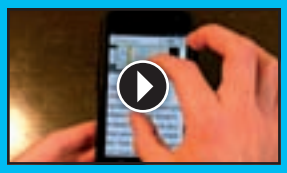
Such a database is called a document-oriented database, and the popular MongoDB, and CouchDB are examples of this kind of database.



AN EXAMPLE OF HOW A GRAPH DATABASE MIGHT REPRESENT A SOCIAL WEBSITE WITH GROUPS

*pointers

>>Nuggets of cool code at work.

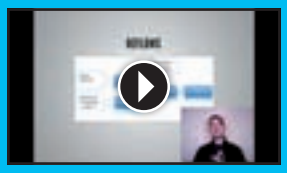


*Chrome for Android

>>Google has released the Android version of its popular Chrome browser. Here's a video that highlights the latest features.
Duration > 06:27
http://dvwx.in/xDAXEQ

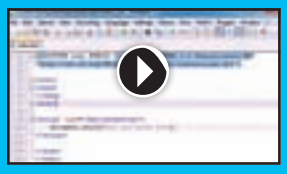
*Firefox Dev tools

>>Firefox 11 (currently on the Aurora channel) features two major new features: the Style Editor and a 3D view for the page inspector ("Tilt").
Duration > 04:09
http://dvwx.in/yT7m6d



*HTML5, CSS3 and DOM performance

>>Paul Irish, from the Chrome team, walks through smart techniques to improve the performance of your app.
Duration > 28:40
http://dvwx.in/AAkAfx



*Beginner JavaScript tutorial

>>Here is an interesting series for those of you who want to get started with the most popular scripting language on the web right now.
Duration > 06:41
http://dvwx.in/wbAnm9

Similarly imagine a Facebook-like application where you have people related to other people, and action performed by one person on another person — Shakeela Swaminathan has poked Prahlad Upadhyay. Here something called a Graph database would be a lot more useful. In a graph databases the people and all their information would be "nodes" with properties such as name, date of birth and other personal details, while edges could connect two nodes, or people; each edge can have its own properties, such as relation, relation start date, poke, etc.

Conclusion

As you can see there is a wealth of ways to represent data that doesn't involve tables with rows and columns, that are linked to further tables of rows and columns. Further, since people are are used to such a model with traditional, SQL-based relational databases there can be a

MULTIPLE COLUMNS THAT SHOW RELATIONS USING COMMON IDS. PURCHASES ARE TIED TO A USER VIA USER_ID. PRODUCTS ARE TIED TO A PURCHASE VIA A PRODUCT_ID

tendency to fit data in this model, even when it does not belong.

Even when such databases are a good fit, it can often be better for performance and scalability that a different model be used.

A lot of the major websites in existence today use such kinds of databases. Facebook engineers developed and open-sourced a non-relational database called Cassandra, which is now part of Apache. Facebook no longer uses it but it is now used by Digg, and even Twitter for some data. Facebook has moved on to a different NoSQL database, HBase. Google uses their own system called BigTable.

Many websites start with SQL-based systems for all their data, and eventually move to NoSQL as it is provide benefits for many kinds of data. So while NoSQL isn't a patch for every SQL problem, such systems provide alternative to certain kinds of data storage, and access patterns. Both NoSQL and SQL databases used together can often provide the balance that many applications need. >